

Predicting NBA Game Winners

Logistic regression, ELO, calibration, and the road to beating the market

Angel Mehmet Can Andrew

Math 17 — Mathematics for Machine Learning

What we want to do

- Given a scheduled NBA game, estimate $p = \Pr(\text{home wins} \mid \text{features})$.
- Build the entire mathematical pipeline from first principles: likelihood $\rightarrow$ loss $\rightarrow$ gradient $\rightarrow$ Hessian $\rightarrow$ optimiser $\rightarrow$ metric.
- Evaluate on the held-out 20% of the 2023–24 and 2024–25 seasons (528 real NBA games).
- Long-term goal: out-predict the prediction-market crowd on log-loss.

Headline result. L2-regularised logistic regression: **67.2% accuracy, log-loss 0.595, AUC 0.740** on a strict chronological hold-out.

The data

Source: `basketball-reference.com` game schedules (regular season + playoffs).

- 2,640 games over 2023–24 and 2024–25
- All 30 NBA franchises
- Home win rate 54.6%
- Chronological 80/20 split: 2,112 train, 528 test

Features used by the LR (10 columns):

- Rolling 10-game differentials of offensive / defensive / net ratings, pace, scoring margin, and win percentage
- Rest-day differential, back-to-back flags
- ELO logit (link to next slide)

All rolling features are shifted by one game to prevent look-ahead leakage; standardisation is fit on the training rows only.

The math: from Bernoulli MLE to BCE

$$p_i = \sigma(x_i^\top w + b), \quad \sigma(z) = \frac{1}{1+e^{-z}}.$$

Bernoulli likelihood $\mathcal{L} = \prod p_i^{y_i} (1 - p_i)^{1-y_i} \implies$ **binary cross-entropy**:

$$J(w, b) = -\frac{1}{n} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

Gaussian prior on the weights $\implies$ L2 penalty ($\lambda = 1/(n\sigma^2)$):

$$J_\lambda(w, b) = J(w, b) + \frac{\lambda}{2} \|w\|_2^2.$$

The loss is not arbitrary. It is the negative log-likelihood under the model, and L2 is the MAP shift under a Gaussian prior.

Gradient, Hessian, convexity

With $p = \sigma(Xw + b\mathbf{1})$:

$$\nabla_w J_\lambda = \frac{1}{n} X^\top (p - y) + \lambda w$$

$$H_{ww} = \frac{1}{n} X^\top \text{diag}(p \odot (1 - p)) X + \lambda I.$$

$p_i(1 - p_i) > 0$, so $X^\top \text{diag}(\cdot) X$ is PSD; with $\lambda > 0$ the Hessian is positive definite. Hence

J_λ is strictly convex with a unique minimiser.

Gradient, Hessian, convexity

With $p = \sigma(Xw + b\mathbf{1})$:

$$\nabla_w J_\lambda = \frac{1}{n} X^\top (p - y) + \lambda w$$

$$H_{ww} = \frac{1}{n} X^\top \text{diag}(p \odot (1 - p)) X + \lambda I.$$

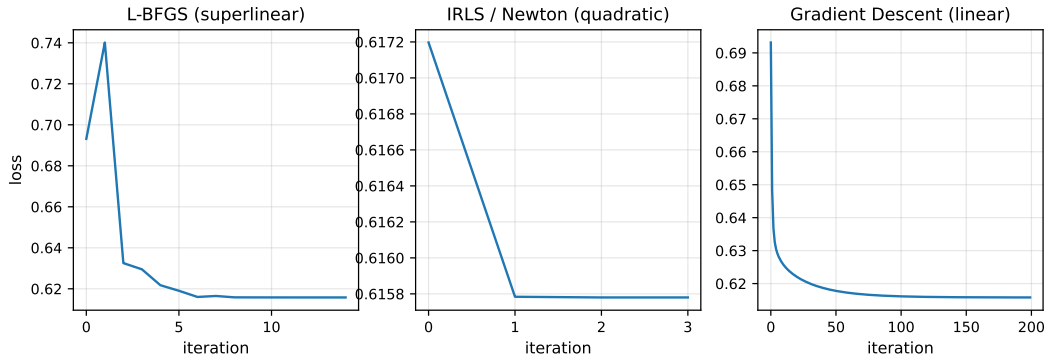
$p_i(1 - p_i) > 0$, so $X^\top \text{diag}(\cdot) X$ is PSD; with $\lambda > 0$ the Hessian is positive definite. Hence

J_λ is strictly convex with a unique minimiser.

Why we care. If every optimiser converges, they converge to the *same* answer. Speed is the only thing that differs.

Three optimisers, one minimum

Same convex loss, three optimisers reach the same optimum



- **L-BFGS** (default): quasi-Newton, superlinear, $O(nd)/\text{step}$.
- **IRLS = Newton**: solves a weighted ridge regression each step; quadratic convergence, $O(nd^2 + d^3)/\text{step}$.
- **Gradient descent**: linear convergence, $O(nd)/\text{step}$.

ELO is logistic regression

Define $\Pr(A \text{ beats } B) = \sigma((R_A - R_B)/s)$ with $s = 400/\ln 10$.

This is identical to $1/(1 + 10^{-(R_A - R_B)/400})$.

So ELO is logistic regression with one feature (the rating difference) and a known scale.

ELO is logistic regression

Define $\Pr(A \text{ beats } B) = \sigma((R_A - R_B)/s)$ with $s = 400/\ln 10$.

This is identical to $1/(1 + 10^{-(R_A - R_B)/400})$.

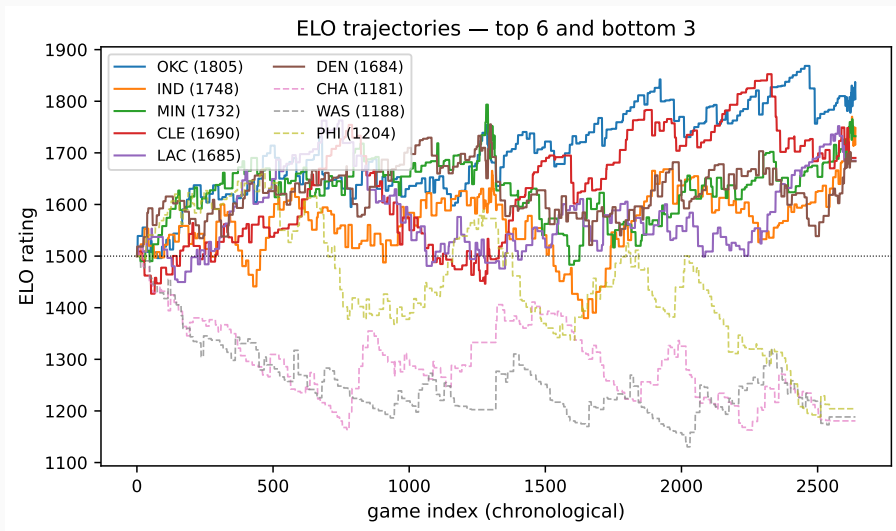
So ELO is logistic regression with one feature (the rating difference) and a known scale.

ELO updates = SGD on log-loss. For one game $\partial L / \partial R_A = (p - y)/s$,
 $\partial L / \partial R_B = -(p - y)/s$. One SGD step with rate Ks :

$$R_A \leftarrow R_A + K(y - p), \quad R_B \leftarrow R_B - K(y - p).$$

The K-factor is the SGD learning rate.

ELO on real data (sanity check)



Adding teams as learned vectors — the NN

- For each team t , learn a vector $e_t \in \mathbb{R}^k$.
- Concatenate $[x; e_h - e_a]$ (*difference* keeps the model invariant under team relabelling — same idea as ELO).
- One ReLU hidden layer, sigmoid head.
- Loss: BCE + L2 on weights + separate L2 on embeddings (Gaussian prior on team strength).
- Optimiser: Adam, early stopping on val.

$$u = [x; e_h - e_a]$$

$$h_1 = \text{ReLU}(W_1 u + b_1)$$

$$z = w_2^\top h_1 + b_2$$

$$p = \sigma(z)$$

*Backprop derivation in full in the module docstring;
verified by a finite-difference test.*

Metrics from first principles

Implemented from scratch in numpy; cross-checked against sklearn in tests.

- **Log-loss** (same formula as training loss) — a proper scoring rule.
- **Brier** $BS = (1/n) \sum (p_i - y_i)^2$ with Murphy's three-way decomposition:

$$BS = REL - RES + UNC.$$

- REL — bin-wise calibration error (lower is better)
 - RES — skill: how much bin frequencies deviate from $\bar{y}$
 - $UNC = \bar{y}(1 - \bar{y})$ — irreducible label variance
- **AUC** via Mann–Whitney U with proper tie handling.
- **ECE** — weighted gap between bin means and bin frequencies.

Calibration (post-hoc)

A model can have high AUC and still be miscalibrated. Two methods:

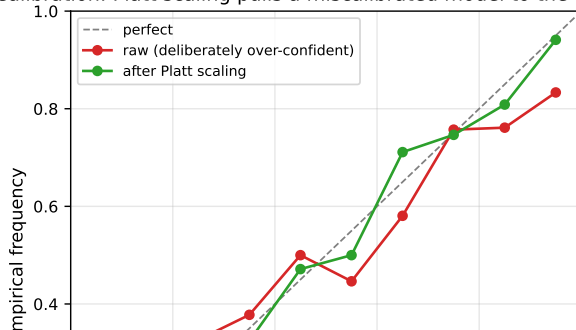
Platt scaling. Fit a 1-D LR on the model's logits: $P(Y = 1|z) = \sigma(Az + B)$.

With Platt smoothing on the targets to avoid overfit on small calibration sets.

Isotonic regression. Best non-decreasing step function g minimising $\sum w_i(g_i - y_i)^2$.

Implemented with *pool-adjacent-violators* in $O(n)$.

Calibration: Platt scaling pulls a miscalibrated model to the diagonal

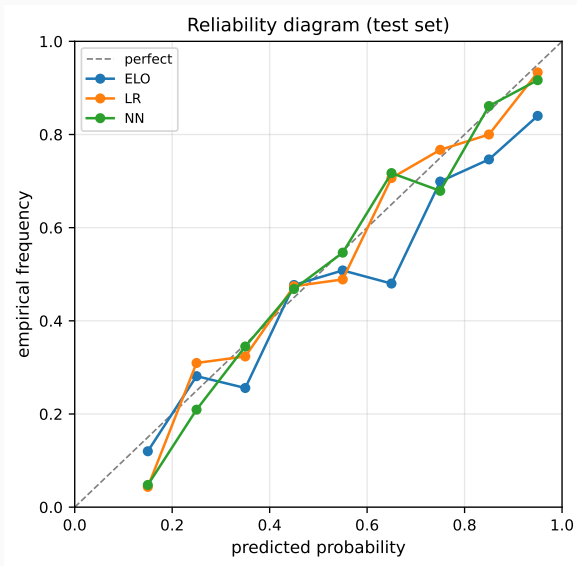


Results — held-out 20% (528 games)

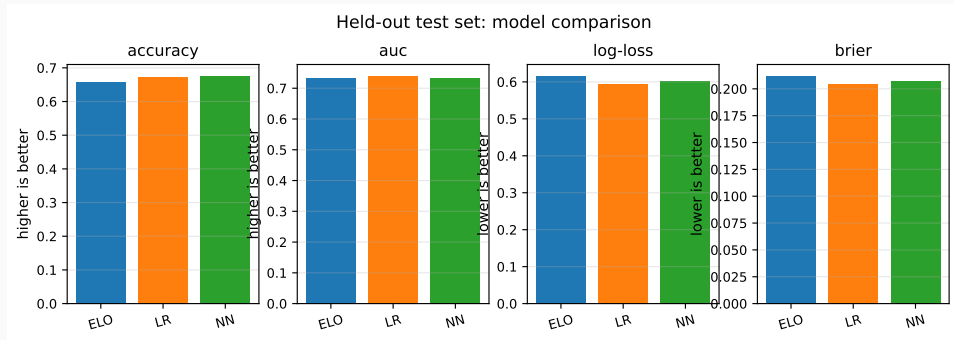
Model	Acc.	Log-loss	Brier	AUC	ECE
ELO	0.659	0.616	0.212	0.734	0.077
LR (L-BFGS)	0.672	0.595	0.205	0.740	0.044
LR + Platt	0.672	0.595	0.205	0.740	0.046
NN (Adam)	0.672	0.602	0.207	0.734	0.041
NN + Platt	0.676	0.601	0.207	0.734	0.040

- LR wins on log-loss, Brier, and AUC.
- NN matches accuracy but doesn't reduce log-loss (small training sample).
- ELO is a strong, free baseline.
- LR's resolution $RES = 0.045$, uncertainty $UNC = 0.248 \Rightarrow$ model explains $\sim 18\%$ of label variance.

Calibration on real data



Side-by-side metrics



What's next

1. **Market comparison.** Code for devigging Kalshi / Polymarket prices, log-loss against the market, and a Kelly backtest is already implemented and tested. We need licensed historical-odds data to evaluate on the same chronological hold-out.
2. **Richer features.** Four-factor counting stats, player availability, road-trip schedule features. The data adapters are stubbed in; the model code accepts them unchanged.
3. **In-game model.** Same architecture, with score differential and seconds remaining; very similar code path.

Limitations / honesty

- Only two seasons of data — $\sim 2,100$ training rows. Deep models cannot win here.
- Without box-score four-factor stats, our offensive / defensive ratings are coarse (we use a 100-possession proxy).
- Recent NBA home-court advantage has fallen; the model learns this from data but it makes the baseline weaker than historical literature would suggest.
- Beating the market on log-loss may genuinely be infeasible on this sample; we will report whatever we find.

`github.com/mehmet-can-yilmaz/nba-predictions`

- Math derivations: `MATH.md`
- Solver implementations: `src/.../logistic_regression.py`, `elo.py`, `neural_net.py`
- Metrics + calibration: `metrics.py`, `calibration.py`
- Market + Kelly: `market.py`
- 30 unit tests pass, including a finite-difference gradient check and a cross-check against `scikit-learn`.

Thank you. Questions?